

Sprint 2 Demo Script — Dartly

Date: April 21, 2026 **Slot:** 15 minutes total (8 min Phase A + 7 min Phase B) **Driver:** Justin (drives the laptop and narrates throughout) **Format:** Justin runs the app; each owner jumps in for their feature during Phase A and for deep technical questions in Phase B.

Pre-Demo Checklist

- ☐ App running at deployed URL or `localhost:3000`
 - ☐ Logged-out state (start fresh)
 - ☐ Seed data loaded:
 - **User A** (`testuser@gmail.com` / `Testpass1!`) — 8+ jobs across every stage, 2+ experiences, 2+ educations, 2+ skills, career prefs filled in, 2+ AI-generated documents linked to a job
 - **User B** (`testuser2@gmail.com` / `Testpass1!`) — ownership denial demo account
 - ☐ CI tab open on the latest green main-branch run
 - ☐ `git log --oneline -10` in a terminal tab (merged S2 PRs visible)
 - ☐ Credentials & test IDs copy-pasted into a scratch file
 - ☐ One job ID noted for the ownership demo in B1
 - ☐ `bun run test` passes locally (133 tests)
-

Speaking Assignments

Section	Time	Driver	Primary Speaker	Jump-in
A1. Dashboard Search / Filter / Sort	2 min	Justin	Justin	—
A2. Job Detail Workflow	2 min	Justin	Kenji	Bruce (stage transition)
A3. Interview / Activity / Outcome	2 min	Justin	Kenji	Bruce (outcome)
A4. Profile + AI Document Generation	2 min	Justin	Francis (profile), Ethan (AI + metrics)	—
B1. Workflow & Data Integrity	2 min	Justin	Bruce	Ethan (version race fix)
B2. CI & Unit Testing	3 min	Justin	Ethan	Bruce
B3. Architecture Q&A	2 min	Justin	Kenji leads; all chime in	—

Justin drives. No laptop switching. Jump-ins are verbal.

Phase A — Scripted Product Demo (8 minutes)

A1. Dashboard Search / Filter / Sort (2 min) — Justin

What to say: "Sprint 2 turned the dashboard from a static list into a real board. Search, filter, sort, and stage badges."

Step	Action	What to show
1	Dashboard loads with 8+ jobs	Stage badges on every card; active count in header
2	Type "Acme" in the search box	Cards filter instantly across title, company, and description
3	Clear search, open the filter dropdown	Filter by stage (Applied, Interview, Offer) — list narrows
4	Switch sort from "Recent" to "Company (A→Z)"	Cards reorder alphabetically
5	Point at stage badges	Each badge color-coded (Interested / Applied / Interview / Offer / Rejected / Archived)
6	Click a priority card	Priority indicator visible on the card

Key line: "Search hits every field. Filter narrows by stage. Sort is local to the current page — server returns all of the user's jobs; filtering and sorting happen client-side for snappy interaction."

A2. Job Detail Workflow (2 min) — Justin drives, Kenji speaks

What to say (Kenji): "Click any card to expand into the full job detail view."

Step	Action	What to show
1	Click any card	Detail view opens with Overview section
2	Point out Overview fields	Title, company, location, compensation, deadline, recruiter/contact notes, custom notes
3	Edit one field (e.g., deadline)	Inline edit, save, toast confirmation
4	Click the stage picker	BRUCE jumps in: "Stage transitions go through a controlled state machine. We enforce valid transitions server-side and write a <code>JobStageHistory</code> row every time — that's where Ethan's response-rate metric gets its data."
5	Move the stage from Applied → Interview	Overview badge updates; <code>lastActivityAt</code> refreshes
6	Refresh the page (F5)	New stage persists

Key line (Kenji): "The detail view is the hub for everything we're about to show — timeline, interviews, follow-ups, documents all live here."

A3. Interview / Activity / Outcome (2 min) — Justin drives, Kenji + Bruce speak

Step	Action	Speaker	What to show
1	Scroll to Activity Timeline	Kenji	Chronological list of stage changes, interviews, follow-ups
2	Click Add Interview	Kenji	Modal for round type (Phone, Technical, Onsite), scheduled time, notes
3	Fill form, save	Kenji	New interview appears on the timeline with round badge
4	Click Add Follow-Up	Kenji	Modal for reminder title + date
5	Save the follow-up	Kenji	Appears on the timeline; <code>completed</code> checkbox visible
6	Click the outcome control in the header	Bruce	Outcome selector: Rejected, Offered, Withdrawn
7	Select "Offered"	Bruce	Badge changes; <code>outcomeAt</code> timestamp persisted

Key line (Kenji): "Timeline, interviews, and follow-ups all hit the same `JobActivity` table — one consistent pattern for anything time-stamped on a job."

A4. Profile + AI Document Generation + Metrics (2 min) — Francis + Ethan

Step	Action	Speaker	What to show
1	Click Profile	Francis	Experience, Education, Skills, Career Preferences — all filled in
2	Click Edit on an experience	Francis	Modal with date picker, "currently working here" toggle, description field
3	Drag to reorder experiences	Francis	Order persists to DB — used by AI resume generation
4	Back to dashboard → open a seeded job → Documents tab	Ethan	Tab shows existing linked documents
5	Click Generate Resume	Ethan	Resume appears; Markdown rendered; version 1 linked to this job
6	Click Generate Cover Letter	Ethan	Cover letter generated using profile + job context
7	Open a draft → click Improve: Make Concise	Ethan	Before/after diff shown; user can accept
8	Back to dashboard → point at Metrics Panel	Ethan	Stage counts, response rate, avg days to response, active applications

Key line (Ethan): "AI routes pull the profile + job into a shared generator, save the result as a Document with version 1, and link it to the job via `JobDocumentLink`. Metrics read from `JobStageHistory` — Bruce's timestamps feed my response-rate math directly."

Phase B — Technical Evidence and Q&A (7 minutes)

B1. Workflow & Data Integrity (2 min) — Bruce speaks; Ethan jumps in

Prep: Have these files open in tabs:

- `src/services/jobs.ts` (stage transition + history write)
- `src/services/documents.ts` (Serializable transaction for version bump)
- `src/tests/services/documents.test.ts` (version race test)

Step	Action	Bruce says	Ethan jump-in
1	Open <code>src/services/jobs.ts</code> around the <code>updateJob</code> function	"When the stage changes, we write a <code>JobStageHistory</code> row inside the same transaction as the job update. Either both land or neither does."	—
2	Show the transaction boundary	"This is why the timeline and metrics never drift from the actual stage — history is a source of truth, not a side effect."	—
3	Open <code>src/services/documents.ts</code> → <code>updateDocumentContent</code>	"Ethan hardened this one mid-sprint — want to walk through it?"	ETHAN: "Two users saving at the same time could read the same max version number. We now do the max-version lookup <i>inside</i> a <code>Serializable</code> transaction and use <code>aggregate</code> . Concurrent writers serialize; no duplicate version rows."
4	Open <code>src/tests/services/documents.test.ts</code> → "computes next version number from <code>max()</code> inside transaction"	—	ETHAN: "This test asserts both the isolation level and the aggregate shape. If someone refactors and drops the transaction wrapper, CI fails."

Key line (Bruce): "Workflow integrity is enforced in the service layer, not the UI. The timeline view is a read of history, not a cache — so the data can't lie."

B2. CI and Unit Testing Evidence (3 min) — Ethan speaks, Bruce jumps in

Step	Action	Ethan says	Bruce jump-in
1	Open GitHub → Actions tab	"Green checks on every recent PR — install, lint, type-check, test, build."	—
2	Click the latest main-branch run	"All 133 tests passing. Up from 60 in Sprint 1."	—
3	Open <code>src/tests/api/documents.test.ts</code>	"Happy path: PUT a new version, get a 200 with <code>versionNumber: 2</code> ."	—
4	Same file, the 404 test	"Negative path: PUT to a non-existent document returns 404, not a stack trace."	—
5	Open <code>src/tests/services/metrics.test.ts</code>	"Dashboard logic test — given a history of 3 APPLIED→INTERVIEW transitions and 5 total applications, <code>responseRate</code> is 60%."	—
6	Open <code>src/tests/services/profile.test.ts</code>	—	BRUCE: "Profile completion logic — <code>computeCompletion</code> returns 100% only when every required section is filled. This test catches anyone who adds a new required field and forgets the completion math."
7	Open <code>src/tests/api/ai-resume.test.ts</code> → the <code>validateBody</code> rejection test	"Non-happy path for AI: if the request body is missing <code>jobId</code> , <code>validateBody</code> throws, and we return a 400 — not a 500 leaking internals."	—

Key stat (Ethan): "133 tests total. 14 new negative-path tests added this week — validation failures, rate-limit 429s, ownership denials, soft-delete filters."

B3. Architecture and Implementation Q&A (2 min) — Kenji leads, team chimes in

Q: How does Sprint 2 extend Sprint 1 without breaking prior flows?

(Kenji) "Every Sprint 2 feature lives in new tables or new columns: `JobStageHistory` , `JobActivity` , `Document` , `DocumentVersion` , `JobDocumentLink` . The Sprint 1 `Job` table gained optional columns (`deadline` , `outcomeAt` , `recruiterNotes`) — all nullable, no migration breakage. All Sprint 1 tests still pass unchanged."

Q: Where does stage-transition logic live, and why?

(Bruce) "In `src/services/jobs.ts` , inside the `updateJob` service function. Two reasons: API routes stay thin and focused on HTTP shape, and the service can be reused by any future caller — a background job, a CLI, a cron — without duplicating the history-write. The transaction boundary is also where it needs to be."

Q: How do timeline, interviews, and documents coordinate inside Job Detail?

(Kenji) "Timeline is a read-only projection over `JobStageHistory` + `JobActivity` ordered by timestamp. Interviews and follow-ups are both `JobActivity` rows with a `type` discriminator — same table, same queries. Documents join through `JobDocumentLink` which points to a specific `DocumentVersion` , so we can pin exactly which draft was sent."

Q: Which profile entities feed the AI, and how?

(Ethan) "AI routes call `getProfile(userId)` → full `ProfileData` with experiences, educations, skills. Combined with `Job{title, company, description}` into a `JobContext` object. Today the generator is a deterministic Markdown template — a stub provider with a `logger.warn` every time it fires. When we swap for a real LLM in Sprint 3, nothing else changes."

Q: What technical debt did Sprint 2 introduce, and how do you plan to pay it down?

(Ethan) "Three things. One: the AI provider is a stub — Sprint 3 has an `AI_PROVIDER` env gate planned. Two: `DocumentVersion` has a Serializable-transaction race guard but no DB-level `@@unique([documentId, versionNumber])` — that needs a coordinated migration because our dev DB is shared. Three: a couple of the Sprint 2 dashboard queries load more columns than they need — I trimmed metrics to `select: { id, stage }` this week; the rest of the dashboard could use the same pass."

Timing Guide

Clock	Section	Notes
0:00	A1 — Search/Filter/Sort	Justin
2:00	A2 — Job Detail	Kenji + Bruce
4:00	A3 — Interview / Activity / Outcome	Kenji + Bruce
6:00	A4 — Profile + AI + Metrics	Francis + Ethan
8:00	Phase B starts	
8:00	B1 — Workflow integrity	Bruce + Ethan
10:00	B2 — CI & Tests	Ethan + Bruce
13:00	B3 — Architecture Q&A	Kenji leads
15:00	Done	

Emergency Fallbacks

Problem	Fallback
App won't load	Have screenshots of every A-section saved; show code and explain
AI generation errors out	Use the pre-seeded documents instead; explain the route flow
Job creation/edit fails	Show the API route + Prisma schema; point at green CI
CI tab unavailable	Show <code>.github/workflows/ci.yml</code> + local <code>bun run test</code> output
Seed data missing	<code>bun run db:seed</code> — takes 15s; or create one job live
Rate limit hits during demo	Wait 60s; use this as the B1 evidence (rate limit is real)

Files to Have Open (Quick Reference)

Evidence	File	Lines
Stage transition + history write	src/services/jobs.ts	updateJob (~80–120)
Document version race fix	src/services/documents.ts	updateDocumentContent (~105–125)
AI route w/ rate limit	src/app/api/ai/resume/route.ts	1–30
Metrics math	src/services/metrics.ts	1–99
Version race test	src/tests/services/documents.test.ts	"computes next version" block
Metrics test	src/tests/services/metrics.test.ts	full file
Profile completion test	src/tests/services/profile.test.ts	computeCompletion block
AI negative path test	src/tests/api/ai-resume.test.ts	"returns 400" / "returns 429" blocks
Ownership denial test	src/tests/api/jobs-id.test.ts	ownership block
CI config	.github/workflows/ci.yml	full file